Data Structures and Algorithms — Lab 6

Topic

Analysis of Singly Linked List Operations in C++ with Practical Demonstrations

Objective

1. Implement basic operations in a Singly Linked List without using a tail pointer.
2. To understand the fundamental structure and components of singly linked lists.
3. To explore practical applications of singly linked lists in real-world programming scenarios.
4. To compare singly linked lists with other linear data structures in terms of flexibility and dynamic memory usage.
5. Develop problem-solving skills by solving questions on paper through dry runs and complexity analysis.

Outcomes

1. Gained a clear understanding of how singly linked lists manage and organize data dynamically.
2. Successfully implemented and tested core operations like insertion, deletion, and traversal.
3. Demonstrated the ability to apply singly linked lists in solving basic programming problems.
4. Developed problem-solving skills by comparing linked lists with other data structures in various use cases.
5. Learn to implement fundamental operations in a Singly Linked List, such as creating nodes and displaying elements, without using a tail pointer.

Content

The following sections will be covered during this lab session.

1. **Singly Linked List Operations**: Implementing basic operations in a singly linked list without using a tail pointer. This includes creating nodes dynamically and displaying the linked list while understanding the time complexity of traversal.
2. **Implementation of Insertion and Deletion Operations**

    - Inserting nodes at the beginning, end, and specific positions
    - Deleting nodes from the beginning, end, and by value

Task Instructions

Students are required to complete the following tasks during lab time.

Create a private repository on your GitHub accounts. The name of the repository should be: **Lab-6-DSA**. All files should be uploaded to this repository; this includes the header and cpp files. Please note that the name of class-related files should be the same as the class name. We can name the main file Task_1.cpp or Task.cpp if there is only 1 task. We recommend you work in .h files for classes only since we must work on templates.

Task 1

Write a C++ program to implement a **Singly Linked** List with the following functionalities:

1. **CreateNode(int value)** → Dynamically creates a new node and adds it to the end of the list.
2. **Display()** → Traverses the list and prints all elements.

Instructions:

- Use a **Class Node** with data and next pointers.
- Implement these functions without using a tail pointer.
- Assume the head pointer is initialized as **NULL/ nullptr.**



Node

Task 2

Implement a singly linked list using Abstract Data Type (ADT) in C++. Include basic operations such as:

- Insert at beginning
- Insert at end
- Delete a node by value
- Traverse and display the list

Task 3

Write a program to implement a singly linked list using functions representing ADT operations. Include a

menu-driven interface for the following operations:

- Insert node at a specific position
- Delete node at a specific position
- Search for an element in the list
- Count total number of nodes

Task 4

A hospital management system needs to maintain a list of patients waiting in line for consultation. Patients arrive at different times and may leave the queue if they cancel their appointment. Using the concept of a singly linked list as an Abstract Data Type (ADT), design and implement a system that supports the following operations:

- Add a new patient to the end of the queue
- Remove a patient from the queue if they cancel
- Display the current list of patients in order
- Count how many patients are currently in the queue

Write a program to simulate this scenario using a singly linked list and demonstrate each operation with sample data.

Task 5

A university library allows students to reserve books that are currently checked out. Each book maintains a **waiting list of students** who wish to borrow it. Students can join the queue, cancel their reservation, or update their priority based on academic urgency (e.g., research, assignment, casual reading). The library must handle multiple reservation queues, one for each book, and ensure students are served fairly and efficiently.

Using a **singly linked list as an Abstract Data Type (ADT)**, design and implement a system that supports the following operations for **each book's reservation queue**:

- Add a new student to the reservation list**, with optional priority tagging**
- Remove a student from the reservation list **if they cancel the request**
- Update a student's priority**, moving them up the queue accordingly**
- Display the current reservation list **for a specific book**
- Count how many students are currently in the reservation list
- Automatically remove the student at the front of the queue **once the book is returned**

Write a program to simulate this scenario using singly linked lists and demonstrate each operation with sample data for at least **two books** and **multiple students**.

Good Luck! Stay focused, think in steps, and enjoy the logic!